

運用 AlloyDB 最新的向量搜尋功能，控管 RAG 品質！

來源：<https://codelabs.developers.google.com/patent-search-rag-recall?hl=zh-tw>

步驟數：9

透過本程式碼研究室，我們將示範使用 AlloyDB 最新 RAG 功能，建立品質受控且符合情境的專利搜尋應用程式！

目錄

1. [總覽](#)
2. [事前準備](#)
3. [資料庫設定](#)
4. [資料擷取](#)
5. [為專利資料建立嵌入](#)
6. [運用 AlloyDB 的新功能執行進階 RAG](#)
7. [使用修改後的查詢和索引參數進行測試](#)
8. [清除所用資源](#)
9. [恭喜](#)

1. 總覽

在不同產業中，情境搜尋都是應用程式的核心功能。檢索增強生成技術採用生成式 AI 輔助的檢索機制，長期以來一直是這項重要技術演進的關鍵推手。生成模型具有大型脈絡窗口和令人驚豔的輸出品質，正在改變 AI。RAG 提供系統化方式，可將背景資訊插入 AI 應用程式和代理程式，讓這些程式/代理程式以結構化資料庫或各種媒體的資訊為基礎。這類情境資料對於釐清事實和確保輸出內容準確性至關重要，但這些結果的準確度如何？您的業務是否高度依賴這些情境比對和關聯性的準確度？那麼這個專案一定會讓您感到有趣！

向量搜尋的隱藏問題不只是建構，而是要瞭解向量比對結果是否真的良好。我們都有過這種經驗，茫然看著一連串結果，心想：「這東西真的有用嗎？」接下來，我們將深入探討如何實際評估向量比對的品質。「那麼，RAG 有什麼變化？」你可能會問。一切！多年來，檢索增強生成 (RAG) 似乎是個有前景但難以實現的目標。現在，我們終於有工具可以建構 RAG 應用程式，並提供重要工作所需的效能和可靠性。

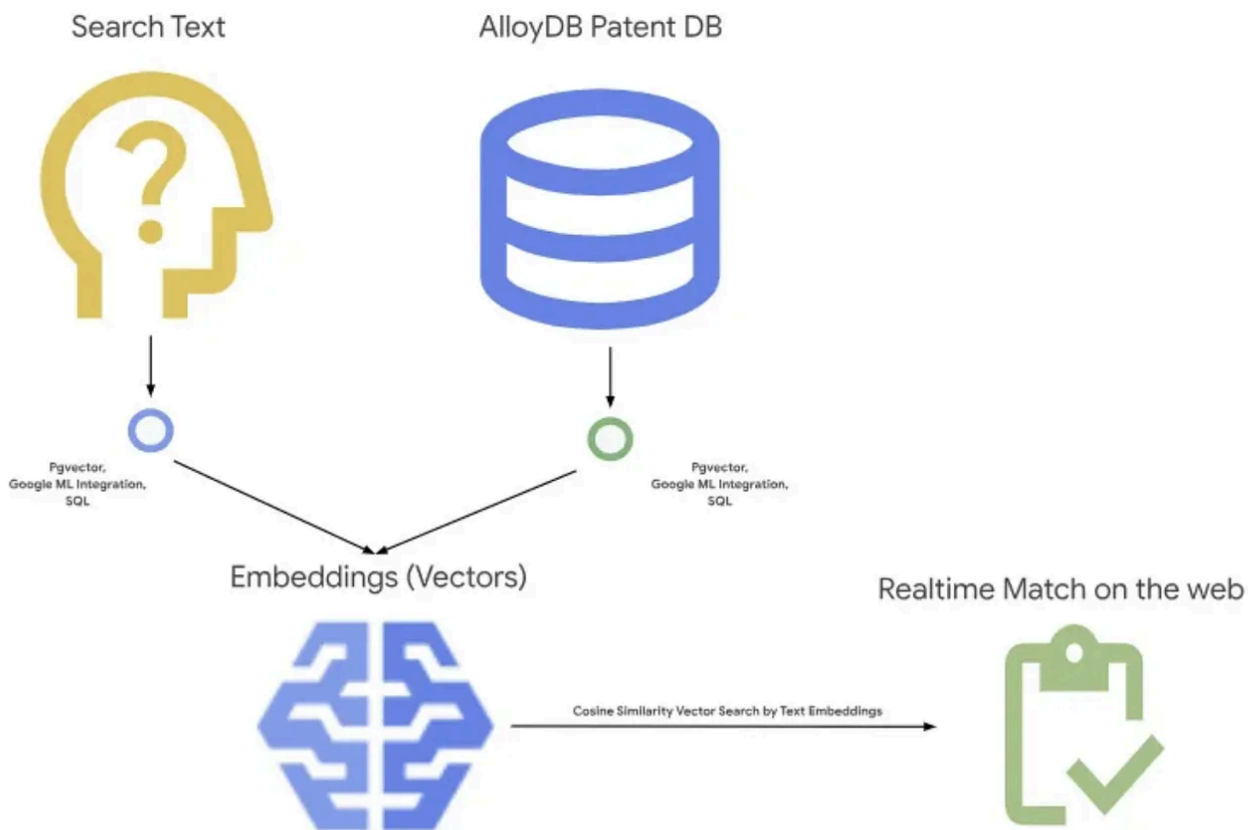
現在我們已初步瞭解以下 3 個主題：

1. 說明脈絡搜尋對代理程式的意義，以及如何使用向量搜尋達成這項功能。
2. 我們也深入探討如何在資料範圍內 (也就是資料庫本身) 取得向量搜尋功能 (如果您還不知道，所有 Google Cloud 資料庫都支援這項功能！)。
3. 我們更進一步，向您說明如何運用 ScaNN 索引支援的 AlloyDB 向量搜尋功能，以高效能和高品質達成這類輕量級的向量搜尋 RAG 功能。

如果您尚未完成這些基礎、中階和稍微進階的 RAG 實驗，建議您依列出的順序閱讀這 3 篇文章、文章和文章。

專利搜尋可協助使用者尋找與搜尋文字相關的專利，我們過去已建構過這項功能的版本。現在，我們將運用全新進階 RAG 功能建構這項應用程式，針對該應用程式進行品質受控的脈絡搜尋。現在就開始吧！

下圖顯示這個應用程式的整體流程。~



目標

使用者可根據文字說明搜尋專利，且搜尋效能和品質都更上一層樓，還能運用 AlloyDB 最新的 RAG 功能評估產生的相符結果品質。

建構項目

本實驗室的學習內容包括：

1. 建立 AlloyDB 執行個體並載入專利公開資料集
2. 建立中繼資料索引和 ScaNN 索引
3. 使用 ScaNN 的內嵌篩選方法，在 AlloyDB 中實作進階向量搜尋
4. 實作 Recall 評估功能
5. 評估查詢回應

需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 已啟用計費功能的 Google Cloud 專案。
-

步驟 2 · 約 0 分鐘

2. 事前準備

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 啟用必要的 API。您可以在 Cloud Shell 終端機中使用 gcloud 指令：

```
gcloud services enable alloydb.googleapis.com compute.googleapis.com  
cloudresourcemanager.googleapis.com servicenetworking.googleapis.com run.googleapis.com  
cloudbuild.googleapis.com cloudfunctions.googleapis.com aiplatform.googleapis.com
```

除了使用 gcloud 指令，您也可以透過控制台搜尋各項產品，或使用這個[連結](#)。

如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

步驟 3 · 約 0 分鐘

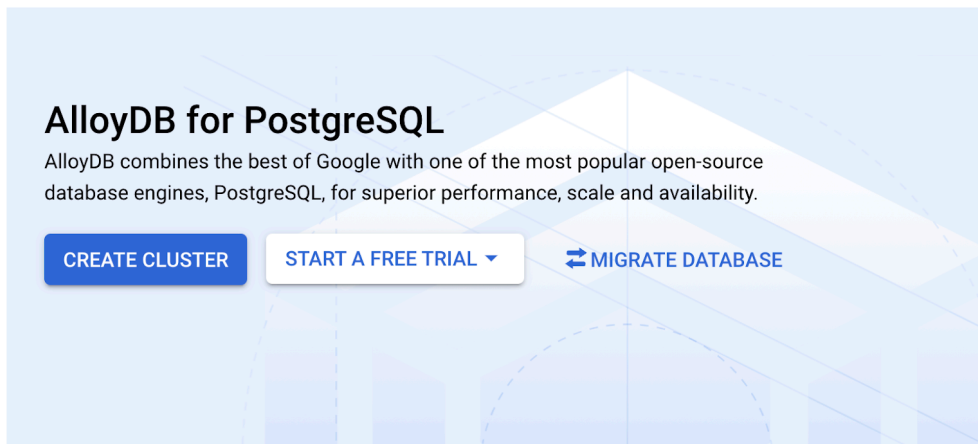
3. 資料庫設定

在本實驗室中，我們將使用 AlloyDB 做為專利資料的資料庫。並使用「叢集」保存所有資源，例如資料庫和記錄檔。每個叢集都有一個「主要執行個體」，可做為資料的存取點。資料表會保存實際資料。

我們來建立 AlloyDB 叢集、執行個體和資料表，以便載入專利資料集。

建立叢集和執行個體

1. 在 Cloud 控制台中前往 AlloyDB 頁面。如要在 Cloud 控制台尋找大部分的頁面，只要使用控制台的搜尋列搜尋即可。
2. 選取該頁面中的「建立叢集」：



3. 畫面上會顯示類似下方的內容。使用下列值建立叢集和執行個體 (如果您要從存放區複製應用程式程式碼，請確保值相符)：

- 叢集 ID：「`vector-cluster`」
- password：「`alloydb`」
- PostgreSQL 15 / 最新建議版本
- Region：「`us-central1`」
- 網路：「`default`」

Configure your cluster

Cluster ID *

Use lowercase letters, numbers, and hyphens. Start with a letter.

Password *



[GENERATE](#)

Set a password for the default "postgres" user. A password is required for the user to log in.

Database version *

PostgreSQL 15 compatible



Location

For better performance, keep your data close to the services that need it. Choice is permanent.

Region *

us-east4 (Northern Virginia)



Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *



[SHOW ADVANCED OPTIONS](#)

4. 選取預設網路後，你會看到如下畫面。

選取「設定連線」。

Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *

default



Private services access connection required

Your network "default" requires a private services access connection. This connection enables your services to communicate exclusively by using internal IP addresses. [Learn more](#)

[SET UP CONNECTION](#)

5. 然後選取「使用系統自動分配的 IP 範圍」並繼續。確認資訊後，選取「建立連結」。

✓ **Enable Service Networking API**

2 **Allocate an IP range**

Google will use this allocated IP range to create subnets.

☐ Select one or more existing IP ranges or create a new one

Select or create an IP range ▼

☒ Use an automatically allocated IP range

Google will automatically allocate an IP range of prefix-length /20 and use the name "default-ip-range".

CONTINUE

3 **Create a connection**

CREATE CONNECTION

CANCEL

6. 設定網路後，即可繼續建立叢集。按一下「CREATE CLUSTER」(建立叢集)，完成叢集設定，如下所示：

Configure your primary instance

Name your instance

Give your instance an ID.

Instance ID *
shopping-cluster-primary

Choice is permanent. Use lowercase letters, numbers, and hyphens. Start with a letter.

Zonal availability

- ☐ Single zone
In case of outage, no failover. Not recommended for production.
- ☒ Multiple zones (Highly available)
Automatic failover to another zone within your selected region. Recommended for production instances. Increases cost.

Machine

AlloyDB's superior performance lets you run your existing workloads on smaller machines.

Machine Type *
8 vCPU, 64 GB

Connectivity

Private IP Connectivity

- ☒ Enable Private IP (always enabled)
Connect from applications and clients within the associated network. [Learn more](#)

CREATE CLUSTER

CANCEL

請務必變更執行個體 ID (您可以在設定叢集 / 執行個體時找到)，然後

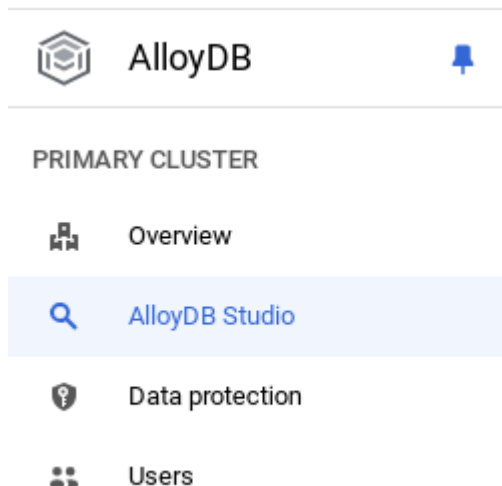
vector-instance。如果無法變更，請記得在所有後續參照中使用執行個體 ID。

請注意，建立叢集約需 10 分鐘。成功後，畫面上會顯示您剛建立的叢集總覽。

步驟 4 · 約 0 分鐘

4. 資料擷取

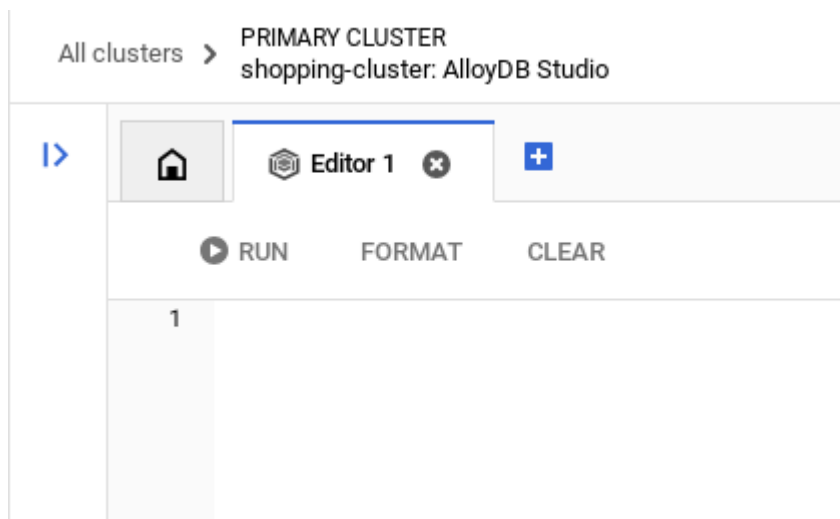
現在要新增包含商店資料的表格。前往 AlloyDB，選取主要叢集，然後選取 AlloyDB Studio：



您可能需要等待執行個體建立完成。完成後，請使用建立叢集時建立的憑證登入 AlloyDB。使用下列資料向 PostgreSQL 進行驗證：

- 使用者名稱：「postgres」
- 資料庫：「postgres」
- 密碼：「alloydb」

成功驗證 AlloyDB Studio 後，即可在編輯器中輸入 SQL 指令。如要新增多個編輯器視窗，請按一下最後一個視窗右側的加號。



您會在編輯器視窗中輸入 AlloyDB 指令，並視需要使用「執行」、「格式化」和「清除」選項。

啟用擴充功能

我們會使用 `pgvector` 和 `google_ml_integration` 擴充功能建構這個應用程式。[pgvector 擴充功能](#)可讓您儲存及搜尋向量嵌入。[google_ml_integration](#) 擴充功能提供多種函式，可存取 Vertex AI 預測端點，並在 SQL 中取得預測結果。執行下列 DDL，[啟用](#)這些擴充功能：

```
CREATE EXTENSION IF NOT EXISTS google_ml_integration CASCADE;  
CREATE EXTENSION IF NOT EXISTS vector;
```

如要查看資料庫已啟用的擴充功能，請執行下列 SQL 指令：

```
select extname, extversion from pg_extension;
```

建立資料表

您可以在 AlloyDB Studio 中使用下列 DDL 陳述式建立資料表：

```
CREATE TABLE patents_data ( id VARCHAR(25), type VARCHAR(25), number VARCHAR(20), country  
VARCHAR(2), date VARCHAR(20), abstract VARCHAR(300000), title VARCHAR(100000), kind  
VARCHAR(5), num_claims BIGINT, filename VARCHAR(100), withdrawn BIGINT, abstract_embeddings  
vector(768)) ;
```

abstract_embeddings 欄可儲存文字的向量值。

授予權限

執行下列陳述式，授予「embedding」函式的執行權：

```
GRANT EXECUTE ON FUNCTION embedding TO postgres;
```

為 AlloyDB 服務帳戶授予 Vertex AI 使用者角色

在 **Google Cloud IAM** 控制台中，授予 AlloyDB 服務帳戶 (格式如下：service-**<<PROJECT_NUMBER>>**@gcp-sa-alloydb.iam.gserviceaccount.com) 「Vertex AI 使用者」角色存取權。PROJECT_NUMBER 會顯示您的專案編號。

或者，您也可以從 Cloud Shell 終端機執行下列指令：

```
PROJECT_ID=$(gcloud config get-value project)  
  
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --  
format="value(projectNumber)")@gcp-sa-alloydb.iam.gserviceaccount.com" \  
  --role="roles/aiplatform.user"
```

將專利資料載入資料庫

我們將使用 BigQuery 上的 [Google 專利公開資料集](#) 做為資料集。我們將使用 AlloyDB Studio 執行查詢。資料會匯入這個 [insert_scripts.sql](#) 檔案，我們會執行這個檔案來載入專利資料。

1. 在 Google Cloud 控制台中，開啟「AlloyDB」[AlloyDB](#) 頁面。
2. 選取新建立的叢集，然後按一下執行個體。
3. 在 AlloyDB 導覽選單中，按一下「AlloyDB Studio」。使用憑證登入。
4. 按一下右側的「新增分頁」圖示，開啟新分頁。

5. 從上述 `insert_scripts.sql` 指令碼複製 `insert` 查詢陳述式，並貼到編輯器。您可以複製 10 到 50 個插入陳述式，快速展示這個用途。
6. 按一下「執行」。查詢結果會顯示在「結果」表格中。

注意：您可能會發現插入指令碼中含有大量資料。這是因為我們在插入指令碼中加入了嵌入內容。如果無法在 GitHub 中載入檔案，請按一下「View Raw」。如果您使用 Google Cloud 的試用抵免額帳單帳戶，為了避免您在後續步驟中產生過多嵌入 (例如最多 20 到 25 個)，因此我們採取這項做法。

步驟 5 · 約 0 分鐘

5. 為專利資料建立嵌入

首先，請執行下列範例查詢，測試嵌入函式：

```
SELECT embedding('text-embedding-005', 'AlloyDB is a managed, cloud-hosted SQL database service.');
```

這應該會傳回查詢中範例文字的嵌入向量，看起來像是浮點數陣列。如下所示：

The screenshot shows a SQL query execution interface. At the top, there are buttons for 'RUN', 'FORMAT', and 'CLEAR'. Below the buttons, the query is displayed: `1 SELECT embedding('textembedding-gecko@001', 'AlloyDB is a managed, cloud-hosted SQL database service.');`. The results section is titled 'RESULTS' and shows a single row of data. The first column is labeled 'embedding' and contains a long list of floating-point numbers. The text 'Query Result for the Embedding Function Call' is displayed at the bottom of the results section.

embedding
-0.029143702,-0.0071177855,-0.024524968,0.053128935,0.02565 5122,-0.04028945,-0.0009277421,0.030361904,-0.017621059,0.00 5031713,0.022722546,-0.00539576,0.044950195,-0.023410132,0.0 5799109,0.030719345,-0.047324233,-0.021004232,-0.039721053,0 017794741,-0.09268643,-0.061628487,0.0009791466,0.01794383,

Query Result for the Embedding Function Call

更新 abstract_embeddings 向量欄位

如果未將 abstract_embeddings 資料插入插入指令碼，請執行下列 DML，將資料表中的專利摘要更新為對應的嵌入：

```
UPDATE patents_data set abstract_embeddings = embedding('text-embedding-005', abstract);
```

如果您使用 Google Cloud 的試用額度帳單帳戶，可能無法產生超過幾個 (最多 20 到 25 個) 嵌入內容。因此，我已在插入指令碼中加入嵌入內容，如果您已完成「將專利資料載入資料庫」步驟，應該會在載入的資料表中看到嵌入內容。

6. 運用 AlloyDB 的新功能執行進階 RAG

現在資料表、資料和嵌入都已準備就緒，讓我們對使用者搜尋文字執行即時向量搜尋。您可以執行下列查詢來測試這項功能：

```
SELECT id || ' - ' || title as title FROM patents_data ORDER BY abstract_embeddings <=>
embedding('text-embedding-005', 'Sentiment Analysis')::vector LIMIT 10;
```

在這項查詢中，

1. 使用者搜尋的文字為「情緒分析」。
2. 我們會在 embedding() 方法中，使用 text-embedding-005 模型將其轉換為嵌入。
3. 「<=>」代表使用餘弦相似度距離法。
4. 我們會將嵌入方法的結果轉換為向量型別，使其與資料庫中儲存的向量相容。
5. LIMIT 10 代表我們選取與搜尋文字最接近的 10 個相符項目。

AlloyDB 可將向量搜尋 RAG 提升至全新境界：

我們推出了許多新功能，以下是其中兩項以開發人員為主的服務：

1. 內嵌篩選
2. 召回評估人員

內嵌篩選

先前開發人員必須執行 Vector Search 查詢，並處理篩選和召回率作業。AlloyDB 查詢最佳化工具會選擇如何執行含篩選器的查詢。內嵌篩選是全新的查詢最佳化技術，可讓 AlloyDB 查詢最佳化工具同時評估中繼資料篩選條件和向量搜尋，並運用向量索引和中繼資料欄的索引。這項功能可提升召回率效能，讓開發人員充分運用 AlloyDB 的現成功能。

內嵌篩選器最適合用於中等選擇性的情況。AlloyDB 搜尋向量索引時，只會計算符合中繼資料篩選條件的向量距離 (查詢中通常在 WHERE 子句中處理的功能篩選器)。這類查詢的效能大幅提升，可補足[篩選後或篩選前](#)的優勢。

1. 安裝或更新 pgvector 擴充功能

```
CREATE EXTENSION IF NOT EXISTS vector WITH VERSION '0.8.0.google-3';
```

如果已安裝 pgvector 擴充功能，請將向量擴充功能升級至 0.8.0.google-3 以上版本，即可使用回想評估工具。

```
ALTER EXTENSION vector UPDATE TO '0.8.0.google-3';
```

只有在向量擴充功能為 <0.8.0.google-3> 時，才需要執行這個步驟。

重要注意事項：如果列數少於 100，就不需要建立 ScaNN 索引，因為這類索引不適用於列數較少的情況。如果是這種情況，請略過下列步驟。

2. 如要建立 ScaNN 索引，請安裝 alloydb_scann 擴充功能。

```
CREATE EXTENSION IF NOT EXISTS alloydb_scann;
```

3. 首先，請不使用索引，且不啟用內嵌篩選器，執行向量搜尋查詢：

```
SELECT id || ' - ' || title as title FROM patents_data
WHERE num_claims >= 15
ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'Sentiment
Analysis')::vector LIMIT 10;
```

結果應如下所示：

```
1 --explain analyze
2 SELECT id || ' - ' || title as title, abstract FROM patents_data
3 where num_claims >= 50
4 ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'sentiment analysis')
::vector LIMIT 10;
```

RESULTS		EXPORT		LEARN
title	abstract			
7813618 - Editing of recorded media	Various embodiments provide a system and meth...			
7809215 - Contextual information encoded in a fo...	Embodiments include a method, a manual device,...			
7181978 - Method, apparatus and system for fore...	A method of forecasting strength of a chemically ...			
8792865 - Method and apparatus for adjusting pa...	Systems and methods are provided for mitigating...			
6235494 - Substrates for assessing mannan-bind...	Provided herein are assays for measuring in vivo l...			
7818325 - Serving geospatially organized flat file ...	A flat file data organization technique is used for ...			
8826453 - Content provider with multi-device sec...	Methods and systems for providing access to co...			
Rows per page: 20 1 - 10 of 10				

4. 對其執行 Explain Analyze (不含索引或內嵌篩選)：

```
1 explain analyze
2 SELECT id || ' - ' || title as title, abstract FROM patents_data
3 where num_claims >= 15
4 ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'sentiment analysis')
::vector LIMIT 10;
```

RESULTS	EXPORT		LEARN
Sort Key: ((abstract_embeddings <=> [-0.007830657,-0.012530749,-0.005363215,-0.009413715,0.077335395...			
Sort Method: top-N heapsort Memory: 42kB			
-> Seq Scan on patents_data (cost=0.00..280.80 rows=1006 width=744) (actual time=0.052..2.210 rows=487 loops=1)			
Filter: (num_claims >= 15)			
Rows Removed by Filter: 483			
Planning Time: 55.481 ms			
Execution Time: 2.422 ms			

執行時間為 2.4 毫秒

5. 我們在 num_claims 欄位上建立一般索引，以便依該欄位篩選：

```
CREATE INDEX idx_patents_data_num_claims ON patents_data (num_claims);
```

6. 讓我們為專利搜尋應用程式建立 **ScaNN** 索引。從 AlloyDB Studio 執行下列指令：

```
CREATE INDEX patent_index ON patents_data
USING scann (abstract_embeddings cosine)
WITH (num_leaves=32);
```

重要注意事項： `(num_leaves=32)` 適用於超過 1000 列的完整資料集。如果資料列數少於 100，就不需要建立索引，因為索引不適用於較少的資料列。

7. 在 ScaNN 索引中啟用內嵌篩選：

```
SET scann.enable_inline_filtering = on
```

8. 現在，我們來執行相同的查詢，但加入篩選條件和向量搜尋：

```
SELECT id || ' - ' || title as title FROM patents_data
WHERE num_claims >= 15
ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'Sentiment
Analysis')::vector LIMIT 10;
```

1 explain analyze
2 SELECT id || ' - ' || title as title FROM patents_data
3 where num_claims >= 15
4 ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'sentiment analysis')::vector LIMIT 10;

RESULTS [EXPORT](#) [SQL](#) [LEARN](#)

Limit (cost=7.95..10.22 rows=10 width=744) (actual time=0.646..0.681 rows=10 loops=1)

-> Index Scan using patent_index on patents_data (cost=7.95..236.26 rows=1006 width=744) (actual time=0.... ✓

Order By: (abstract_embeddings <=> '[0.007830657,-0.012530749,-0.005363215,-0.009413715,0.077335395,... ✓

Filter: (num_claims >= 15)

Rows Removed by Filter: 1

Planning Time: 127.298 ms

Execution Time: 0.747 ms

如您所見，相同的向量搜尋執行時間大幅縮短。Vector Search 上的內嵌篩選 ScaNN 索引，讓這一切成為可能！

接著，我們來評估啟用 ScaNN 的向量搜尋功能召回率。

召回評估人員

相似性搜尋的召回率是指從搜尋中擷取的相關例項百分比，也就是真陽性數。這是最常用的搜尋品質評估指標。召回率損失的其中一個來源，是近似最鄰近搜尋 (ANN) 與 k (精確) 最鄰近搜尋 (kNN) 之間的差異。AlloyDB 的 ScaNN 等向量索引會實作 aNN 演算法，讓您加快大型資料集的向量搜尋速度，但召回率會略有損失。現在，AlloyDB 可讓您直接在資料庫中，針對個別查詢評估這項取舍，並確保長期穩定。您可以根據這項資訊更新查詢和索引參數，以獲得更出色的結果和效能。

搜尋結果的召回邏輯為何？

在向量搜尋的脈絡中，召回率是指索引傳回的向量中，屬於真正最鄰近向量的百分比。舉例來說，如果 20 個最鄰近項目的查詢傳回 19 個最鄰近的基準真相項目，則召回率為 $19/20 \times 100 = 95\%$ 。召回率是搜尋品質的指標，定義為客觀上最接近查詢向量的傳回結果所占百分比。

您可以使用 `evaluate_query_recall` 函式，找出特定設定的向量索引向量查詢召回率。您可以使用這個函式調整參數，以取得所需的向量查詢召回結果。

重要注意事項：

如果在下列步驟中，HNSW 索引發生權限遭拒錯誤，請暫時略過整個召回評估部分。這可能是因為本程式碼研究室撰寫時，這項功能才剛發布，因此存取權受到限制。

- 1. 在 ScaNN 索引和 HNSW 索引上設定「啟用索引掃描」旗標：

```
SET scann.enable_indexscan = on
SET hnswe.enable_index_scan = on
```

- 2. 在 AlloyDB Studio 中執行下列查詢：

```
SELECT
  *
FROM
  evaluate_query_recall($$
    SELECT
      id || ' - ' || title AS title,
      abstract
    FROM
      patents_data
      where num_claims >= 15
    ORDER BY
      abstract_embeddings <=> embedding('text-embedding-005',
        'sentiment analysis')::vector
    LIMIT 25 $$,
    '{"scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10}',
    ARRAY[ 'scann' ] );
```

`evaluate_query_recall` 函式會將查詢做為參數，並傳回查詢的召回率。我使用與檢查效能時相同的查詢做為函式輸入查詢。我已將 ScaNN 新增為索引方法。如需更多參數選項，請參閱[說明文件](#)。

我們使用的這項向量搜尋查詢的召回率：

RESULTS EXPORT [] v LEARN

id	query	configurations	recall	ann_execution_time
1	SELECT id ' - ' title AS title, abstract FROM patents_data where num_claims >= 15 ORDER BY	{"scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10, "hnswe.ef_search": 1}	0.7	93.12

我看到 RECALL 為 70%。現在我可以使用這項資訊變更索引參數、方法和查詢參數，並改善這項向量搜尋的召回率！

7. 使用修改後的查詢和索引參數進行測試

現在，請根據收到的召回通知修改查詢參數，測試查詢。

1. 我將結果集中的資料列數修改為 7 (先前為 25)，發現 RECALL 提升至 86%。

```
7 | id || ' - ' || title AS title,
8 | abstract
9 | FROM
10 | patents_data
11 | where num_claims >= 15
12 | ORDER BY
13 | abstract_embeddings <=> embedding('text-embedding-005',
14 | 'sentiment analysis')::vector
15 | LIMIT 7 $$,
16 | '{"scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10, "hns
ef search":1}'
```

RESULTS

d	query	configurations	recall
1	SELECT id ' - ' titl...	{'scann.num_leaves_to_search':1, 'scann...	0.8571428571428571

也就是說，我可以根據使用者的搜尋情境，即時調整使用者看到的相符結果數量，提高相符結果的關聯性。

2. 請修改索引參數，然後再試一次：

在這項測試中，我將使用「L2 距離」，而非「餘弦」相似度距離函式。我還會將查詢限制變更為 10，展示即使搜尋結果集數量增加，搜尋結果品質是否仍有提升。

[BEFORE] 查詢，使用餘弦相似度距離函式：

```
SELECT
*
FROM
  evaluate_query_recall($$
SELECT
  id || ' - ' || title AS title,
  abstract
FROM
  patents_data
  where num_claims >= 15
ORDER BY
  abstract_embeddings <=> embedding('text-embedding-005',
  'sentiment analysis')::vector
LIMIT 10 $$,
  '{"scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10}',
  ARRAY['scann']);
```

非常重要的附註：你可能會問：「我們怎麼知道這個查詢使用餘弦相似度？」您可以使用「<=>」代表餘弦距離，藉此識別距離函式。

Vector Search 距離函式的[說明文件連結](#)。

上述查詢的結果如下：

		RUN		FORMAT	CLEAR			Valid
8	FROM							
9	patents_data							
10	where num_claims >= 15							
11	ORDER BY							
12	abstract_embeddings <=> embedding('text-embedding-005',							
13	'sentiment analysis')::vector							
14	LIMIT 10 \$\$,							
15	'{"scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10}',							
16	ARRAY['scann']);							
17								
RESULTS								
EXPORT								
LEARN								
id	query	configurations	recall	ann_execution_time	ground_truth_execution_time	index_type		
1	SELECT id ' - ' title AS title, abstract FROM patents_data where num_claims >= 15 ORDER BY	{'scann.num_leaves_to_search':1, "scann.pre_reordering_num_neighbors":10}	0.7	86.097	85.222	scann		

如您所見，在未變更任何索引邏輯的情況下，RECALL 為 **70%**。還記得我們在「內嵌篩選」一節的步驟 6 中建立的 Scann 索引「`patent_index`」嗎？執行上述查詢時，這個索引仍然有效。

現在，我們來建立使用不同距離函式查詢的索引：**L2 距離**：`<->`

```
drop index patent_index;

CREATE INDEX patent_index_L2 ON patents_data
USING scann (abstract_embeddings L2)
WITH (num_leaves=32);
```

刪除索引陳述式只是為了確保資料表上沒有不必要的索引。

現在，我可以執行下列查詢，評估變更向量搜尋功能距離函式後的 RECALL。

[AFTER] 查詢，使用餘弦相似度距離函式：

```
SELECT
*
FROM
evaluate_query_recall($$
SELECT
id || ' - ' || title AS title,
abstract
FROM
patents_data
where num_claims >= 15
ORDER BY
abstract_embeddings <-> embedding('text-embedding-005',
'sentiment analysis')::vector
LIMIT 10 $$,
'{"scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10}',
ARRAY[ 'scann' ]);
```

上述查詢的結果如下：

▶ RUN SELECTED

FORMAT

CLEAR

Valid

```
7  -- FROM
8  FROM
9  patents_data
10 where num_claims >= 15
11 ORDER BY
12 abstract_embeddings <-> embedding('text-embedding-005',
13 'sentiment analysis')::vector
14 LIMIT 10 $$,
15 '{ "scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10}',
16 ARRAY['scann']);
```

RESULTS

EXPORT [] LEARN

id	query	configurations	recall	ann_execution_time	ground_truth_execution_time	index_type
1	SELECT id ' - ' title AS title, abstract FROM patents_data where num_claims >= 15 ORDER BY abstract_embeddings	{ "scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10}	0.9	90.203	107.296	scann

回想價值的轉變，90%！

您也可以根據所需的召回值和應用程式使用的資料集，變更索引中的其他參數，例如 num_leaves 等。

8. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 前往 Google Cloud 控制台的[資源管理員](#)頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
 4. 或者，您也可以點選「DELETE CLUSTER」按鈕，刪除我們剛為這個專案建立的 [AlloyDB 叢集](#) (如果您在設定叢集時未選擇 us-central1，請變更這個超連結中的位置)。
-

步驟 9 · 約 0 分鐘

9. 恭喜

恭喜！您已成功運用 AlloyDB 的進階向量搜尋功能，建構脈絡專利搜尋查詢，不僅效能優異，還能真正以意義為導向。我已整合多種工具，並透過 ADK 和我們在此討論的所有 AlloyDB 內容，建立品質受控的代理應用程式，打造高效能的專利向量搜尋與分析代理，您可以在這裡觀看：<https://youtu.be/Y9fvVY0yZTY>

如要瞭解如何建構該代理程式，請參閱這項[程式碼研究室](#)。

[立即開始使用！](#)
